

AUDITORIA DE SEGURANÇA

Relatório de Auditoria de Segurança

Armazix — plataforma de e-commerce e PDV multi-tenant

Data	28 de agosto de 2026
Escopo	Repositório principal Armazix (backend Cloudflare Workers + frontend TanStack Start)
Metodologia	5 categorias mapeadas à stack detectada — ver nota metodológica
Achados	5 verificados no código real

Nota metodológica

As cinco categorias solicitadas foram mapeadas para a stack real detectada neste repositório antes do início da auditoria. A stack identificada:

Linguagem	TypeScript
Runtime	Cloudflare Workers (edge), compilado com Vite
Framework	TanStack Start (React 19) + TanStack Router — SSR/SPA híbrido
ORM / Banco	Drizzle ORM sobre PostgreSQL (Neon, driver serverless HTTP + WebSocket Pool)
Autenticação	JWT próprio (jose, HS256) + cookie HttpOnly/Secure/SameSite=Strict; senha via PBKDF2-SHA-256 (bcryptjs como fallback legado)
Frontend	React + Tailwind CSS + Radix UI, mesma base TanStack Start do backend (rotas de arquivo)
Deploy	Cloudflare Workers via Wrangler (wrangler.jsonc) — sem Docker/Helm/Terraform em produção; docker-compose.dev.yml só para Redis local de desenvolvimento
Cache	Upstash Redis (REST, opcional — fail-safe para o banco se ausente)
Pagamentos	Mercado Pago (checkout de assinatura da plataforma + checkout de pedidos por loja)

Como cada categoria foi mapeada e verificada:

Banco sem tranca (isolamento de tenant). Não é Supabase/RLS-first: o projeto usa Postgres via Drizzle com DOIS mecanismos coexistindo — filtro manual eq(storeId) em cada query (padrão dominante) e Row Level Security real do Postgres via role restrita armazix_tenant + set_config('app.current_store_id', ...) dentro de transações explícitas (padrão usado nos fluxos mais críticos: criação de pedido, mudança de status, fechamento de encomenda no PDV, aplicação de resultado de pagamento). Auditado verificando se toda query de listagem/agregação/relatório inclui o filtro por storeId derivado do JWT.

Permissão definida no navegador. Cruzamento manual entre os gates de UI do painel admin (checagens de role/permissão em componentes e rotas) e o handler de backend correspondente, verificando se este último reexecuta uma checagem equivalente (requireStoreAccess vs. requireStoreOwner vs. nenhuma checagem de papel).

IDOR. Todos os 35 arquivos de handler em src/lib/api (167 funções exportadas, mapeadas 1:1 às ~150 rotas registradas em src/lib/api-handler.ts) foram percorridos, com atenção a toda operação que recebe um ID por path, query ou corpo e o usa numa cláusula WHERE, SELECT, UPDATE ou DELETE.

Chaves expostas. Grep por padrões de segredo (chaves/tokens/senhas hardcoded, prefixos conhecidos de provedores) em todo o código-fonte, arquivos de config (wrangler.jsonc, .env.example, docker-compose.dev.yml) e em todo o histórico git (git log -p -S) — não só o HEAD atual.

Inputs sem tratamento (XSS). Grep por dangerouslySetInnerHTML/innerHTML/eval/new Function no frontend, inspeção de todo href/src dinâmico, e leitura de todos os templates de HTML gerados no backend (e-mails transacionais) em busca de interpolação de dado de usuário sem escape.

Resumo executivo

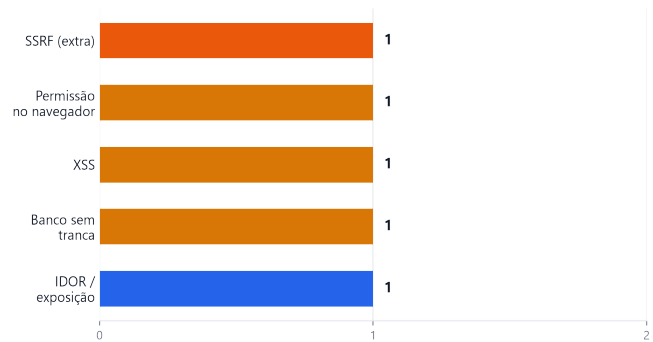


A auditoria percorreu sistematicamente os 35 arquivos de handler do backend (167 funções exportadas) e o frontend administrativo em busca de falhas nas 5 categorias solicitadas. O resultado é um sistema com uma postura de segurança consistentemente madura — isolamento de tenant, prevenção de IDOR e validação de webhook de pagamento em nível acima da média — com 2 falhas concretas e acionáveis (gate de permissão dos Relatórios e HTML injection em e-mail de convite), 1 falha de alta severidade fora do escopo original (SSRF no teste de impressora, reportada por completude), e 2 observações arquiteturais/residuais de menor urgência.



Alta (1)
Média (3)
Baixa (1)

Achados por severidade



Achados por categoria

Pontos fortes verificados

O que segue foi lido linha a linha no código real (não inferido) e confirma proteção efetiva — evidência de cobertura da auditoria, não apenas ausência de achados.

- **IDOR — padrão "verificar-antes-de-agir" consistente**

`src/lib/api/crud-handler.ts` (produtos, categorias, pedidos, cupons, clientes) e praticamente todos os outros handlers de escrita confirmam posse do registro (`and(eq(id), eq(storeId))`) antes de `UPDATE/DELETE`, com comentário `"// IDOR Fix"` explícito em cada ponto. Nenhuma exceção real encontrada em ~20 handlers lidos linha a linha.

- **storeId nunca sai do corpo/query em rota protegida**

`src/lib/api-handler.ts:421-425` — `getRequestBodyStoreId()` sempre retorna null, com comentário reforçando que o `storeId` vem só do JWT. `requireStoreAccess()` (`src/lib/auth/require-store-access.ts`) é a única fonte de verdade e sempre confere a tabela `storeUsers` no banco.

- **Webhooks de pagamento nunca confiam no corpo recebido**

`src/lib/api/payment-handler.ts` (`mp-webhook`) e `src/lib/api/subscription-handler.ts` (`pix/assinatura`) sempre buscam o pagamento real na API do Mercado Pago com o token da própria loja, cross-checkam `storeId`, moeda e valor pago $\geq$ devido, e são idempotentes (não reprocessam o mesmo `payment id`, não fazem downgrade de `"paid"`).

- **Escrita em identidade global de usuário multi-loja**

`src/lib/api/user-handler.ts` — `checkMember()/exclusive` bloqueia um dono de loja de alterar senha/dados globais de um funcionário que também atua em outra loja, cobrindo um caso de `tenant-isolation` que a maioria dos produtos SaaS nem considera.

- **Senhas e sessão**

PBKDF2-SHA-256 (100k iterações) com fallback `bcrypt` só para hashes legados, comparação em tempo constante, política de senha forte, `sessionVersion` incrementado a cada troca de senha/desativação (derruba sessões já emitidas na hora, não só bloqueia login novo).

- **Segredos de aplicação**

`requireJwtSecret()` e toda leitura de `ENCRYPTION_KEY` falham explicitamente (erro 500 visível) se a variável não estiver configurada — nenhum fallback default silencioso encontrado em nenhum dos dois.

- **Nenhum segredo real no código-fonte ou no histórico git**

`.env` nunca foi commitado (confirmado via `git log --all`); `.env.example` só tem placeholders (senha, `seu_token_upstash_aqui`, etc.); `wrangler.jsonc` não grava nenhum valor sensível, só documenta quais secrets configurar via `wrangler secret put`.

- **Convites de equipe**

`src/lib/api/store-invite-handler.ts` — token hashado antes de gravar/comparar no banco (nunca em texto puro), de uso único (`acceptedAt`) e expira em 7 dias.

- **mock-login isolado de produção**

`src/lib/api/auth/mock-login-handler.ts` retorna 404 quando `NODE_ENV !== "development"`; `NODE_ENV` é fixado `"production"` em `wrangler.jsonc` (vars), um valor de deploy, não algo que uma requisição possa influenciar.

- **XSS — nenhuma superfície viva encontrada**

Nenhuma lib de markdown no projeto. Os 2 únicos `dangerouslySetInnerHTML` do repositório são: (a) `components/ui/chart.tsx` — componente `shadcn` não importado em lugar nenhum do app (código morto); (b) `__root.tsx` — script de tema 100% estático, sem interpolação de dado de usuário. Todo href dinâmico voltado a WhatsApp/Maps usa esquema fixo `https://` com dado normalizado (só dígitos no telefone).

- **Gestão de perfis de acesso**

`src/lib/api/permissions-handler.ts` usa `requireStoreOwner` (não só `requireStoreAccess`) nas 4 rotas, e faz allowlist explícita das chaves de permissão aceitas — o corpo da requisição nunca grava uma chave desconhecida.

Pontos fracos — riscos centrais

Ordenados por severidade. Detalhamento completo (arquivo, linha, trecho de código, condições de exploração) na seção seguinte.

ALTA

F1 — SSRF autenticado no teste de impressora de rede
src/lib/api/print-handler.ts

MÉDIA

F2 — Gate de permissão dos Relatórios é decorativo — sem checagem equivalente no backend
src/routes/admin/relatorios.tsx · src/lib/api/reports-handler.ts

MÉDIA

F3 — HTML injection em e-mails transacionais (convite de equipe)
src/lib/auth/email.ts · src/lib/api/store-invite-handler.ts · src/lib/api/store-handler.ts

MÉDIA

F4 — Cobertura de RLS real inconsistente — a maioria dos handlers depende só do filtro manual
src/lib/db/index.ts · src/lib/api/pdv-handler.ts

BAIXA

F5 — Oráculo de enumeração de PII por telefone no checkout (mitigado, residual)
src/lib/api/crud-handler.ts

Achados detalhados

F1 — SSRF autenticado no teste de impressora de rede

ALTA

Categoria	Achado adicional (fora das 5 categorias — rede/SSRF)
Arquivo	src/lib/api/print-handler.ts
Linha(s)	65–120 (núcleo: 104–107)

```
export async function printRawTestHandler(request, auth) {
  try { await requireStoreAccess(auth); } catch (error) { ... }
  const body = await request.json(); // { path, columns?, type?, layout? }
  ...
  const net = parseNetworkPath(body.path.trim());
  if (net) {
    await sendViaTcp(net.host, net.port, escpos); // conecta em QUALQUER host:porta
  }
}
```

Descrição

POST /api/printers/test-raw aceita qualquer usuário autenticado com acesso a QUALQUER loja (requireStoreAccess não distingue papel — um "operador" serve). body.path é validado só quanto ao formato (IPv4 ou hostname com ponto) — não quanto ao destino: não há bloqueio de IPs privados/loopback/link-local nem allowlist de hosts. O servidor então abre uma conexão TCP bruta (via net.createConnection, habilitado pelo compatibility_flags nodejs_compat do Worker) para esse host:porta e escreve os bytes de um cupom ESC/POS de amostra.

Por que é explorável

Um usuário autenticado com qualquer papel na loja (o menor privilégio já serve) pode fazer o backend do Armazix abrir conexões TCP para endereços arbitrários — inclusive faixas internas (10.0.0.0/8, 169.254.169.254, etc.) — e observar no campo `error`/`sent` da resposta se a conexão conectou, deu timeout (5s) ou foi recusada. Isso é um oráculo clássico de SSRF/port-scan: dá para varrer a rede que o Worker enxerga, uma porta por chamada, até 60 vezes por minuto (tier de rate limit "api" padrão, já que esta rota não tem uma entrada específica em rateLimitConfigs). O mesmo primitivo é alcançável em dois passos por printers.ts: criar uma impressora com path arbitrário (createPrinterHandler não valida o campo path) e depois chamar /api/printers/print-test ou /api/printers/print-order com send:true.

Condições de exploração

Requer apenas uma sessão autenticada com acesso a alguma loja (qualquer papel — não precisa ser dono/admin). Não depende de nenhuma feature flag; o compatibility_flags nodejs_compat que habilita net.createConnection já está ativo em wrangler.jsonc.

Impacto

Reconhecimento/varredura da rede interna acessível ao Worker; potencial abuso como relay para sondar ou incomodar hosts de terceiros usando a infraestrutura da Armazix como origem. Não expõe dados de outras lojas diretamente (o payload ESC/POS é fixo, não é o corpo da requisição), mas é uma primitiva de rede real com múltiplos objetivos de abuso.

Recomendação

Validar body.path/printer.path contra uma allowlist de faixas de IP privadas conhecidas do parque de impressoras (ex.: só RFC1918 192.168.0.0/16, 10.0.0.0/8 range documentado pela loja) OU, no mínimo, bloquear explicitamente loopback (127.0.0.0/8), link-local (169.254.0.0/16 — inclui o metadata endpoint de várias clouds), multicast e faixas reservadas antes de chamar sendViaTcp. Adicionar rate-limit dedicado (tier restritivo) para as 3 rotas de impressão em rateLimitConfigs.

F2 — Gate de permissão dos Relatórios é decorativo — sem checagem equivalente no backend

MÉDIA

Categoria	1. Permissão definida no navegador
Arquivo	src/routes/admin/relatorios.tsx · src/lib/api/reports-handler.ts
Linha(s)	relatorios.tsx:88, 60, 64, 73 · reports-handler.ts (todos os 8 handlers)

```
// relatorios.tsx:88
function usePermissaoUsuario(): Permissao { return "admin"; }

// relatorios.tsx:73 – catálogo declara a intenção...
{ id: "aud-002", nome: "Logs de Alterações Críticas", ..., permissao: ["admin"] }

// reports-handler.ts – ...mas o backend nunca checa
async function resolveStoreId(auth) {
  return { storeId: (await requireStoreAccess(auth)).storeId };
// sem checagem de papel/permissão nenhuma
}
```

Descrição

O catálogo de relatórios declara explicitamente quais papéis podem ver cada relatório (permissao: ["admin"] para Logs de Alterações Críticas; ["admin","gerente"] para Lucro Bruto e Líquido; ["admin","gerente","financeiro"] para Fluxo de Caixa), mas usePermissaoUsuario() — a função que deveria ler o papel real do usuário logado — está implementada como um stub que sempre devolve "admin". O filtro de UI nunca esconde nada. E os 8 handlers reais em reports-handler.ts (estoque-baixo, clientes-top, produtos-lucrativos, vendas-periodo, fluxo-caixa, lucro-bruto-liquido, logs-criticos, categorias-financeiro) só chamam requireStoreAccess — que aceita qualquer papel de storeUsers —, nunca uma checagem de papel/permissão equivalente à declarada no catálogo.

Por que é explorável

Qualquer funcionário autenticado da loja — inclusive o papel mais baixo ("operador") — pode chamar GET /api/reports/logs-criticos ou GET /api/reports/fluxo-caixa diretamente (sem passar pela tela) e receber fluxo de caixa, margens de lucro e o log de auditoria de ações críticas (estornos, exclusões, ajustes de estoque) de toda a equipe — dado que o próprio produto já modela como restrito a admin/gerente/financeiro.

Condições de exploração

Basta ter uma conta de qualquer papel na loja (vendedor, operador, etc.); não precisa de nenhuma configuração especial. É isolamento entre papéis dentro da MESMA loja — não é um vazamento entre lojas diferentes.

Impacto

Exposição de dados financeiros sensíveis (fluxo de caixa, lucro líquido) e do log de auditoria de ações críticas para funcionários que o dono da loja não pretendia que tivessem esse acesso.

Recomendação

Implementar usePermissaoUsuario() para ler o storeRole real (já disponível via /api/store-users/list ou embutível no JWT) em vez do stub fixo, E adicionar a mesma checagem de papel dentro de cada handler em reports-handler.ts (reaproveitando o sistema de roleProfiles/ALL_PERMISSION_KEYS que já existe em permissions-handler.ts para outras áreas do produto) — a checagem de UI sozinha nunca é suficiente.

F3 — HTML injection em e-mails transacionais (convite de equipe)

MÉDIA

Categoria	5. Inputs sem tratamento (XSS)
Arquivo	src/lib/auth/email.ts · src/lib/api/store-invite-handler.ts · src/lib/api/store-handler.ts
Linha(s)	email.ts:108 (template) · store-invite-handler.ts:82 · store-handler.ts:294

```
// email.ts:108 – interpolação direta em HTML, sem escape
<p>Olá, <strong>${name}</strong>! A loja <strong>${storeName}</strong> convidou você...</p>

// store-invite-handler.ts:82 – nome do convite só tem trim(), sem sanitizeString()
const name = body.name?.trim();

// store-handler.ts:294 – nome da loja gravado sem sanitização
.set({ name: body.name, ... })
```

Descrição

verificationTemplate/teamInviteTemplate/passwordResetTemplate (email.ts) constroem o HTML do e-mail via template literal, interpolando name/storeName sem nenhum escape de caracteres HTML. No fluxo de cadastro (register-handler.ts) o nome passa por sanitizeString() (remove < e >) antes de chegar ao e-mail, mas dois outros caminhos que alimentam o MESMO template não passam por sanitização nenhuma: (1) o campo "nome" do convite em inviteStoreUserHandler só recebe .trim(); (2) o nome da loja pode ser alterado a qualquer momento em Configurações (updateStoreHandler) gravando body.name direto no banco, sem sanitização.

Por que é explorável

Um dono/admin de loja (autenticado, requireStoreOwner) pode: (a) definir o nome da própria loja como HTML/CSS malicioso via /api/store/update, e/ou (b) convidar um "novo funcionário" via /api/store-users/invite com name="" ou uma estrutura HTML que sobrescreve o botão "Aceitar convite" por um link de phishing. Em ambos os casos, um e-mail real, enviado pelo domínio armazix.com.br via Resend, chega para um TERCEIRO — o convidado, que pode nem ter conta ainda — com esse HTML não escapado embutido.

Condições de exploração

Requer uma conta de dono/admin de alguma loja (não é anônimo), mas a vítima é um terceiro que não precisa ter conta nem qualquer relação prévia com quem envia o convite.

Impacto

Phishing/defacement dentro de um e-mail legítimo enviado pelo domínio da própria Armazix — o vetor mais crítico de execução de script depende do cliente de e-mail do destinatário (a maioria bloqueia <script>/onerror em 2026), mas a estrutura HTML/CSS do e-mail pode ser reescrita para convencer o destinatário a clicar num link diferente do botão real.

Recomendação

Fazer HTML-escape (&, <, >, ", ') de todo valor interpolado nos três templates de email.ts antes de embuti-lo — não depender de sanitização feita no ponto de entrada, já que há mais de um caminho até o mesmo template. Aplicar sanitizeString() (ou um escapador de HTML dedicado) em inviteStoreUserHandler e em updateStoreHandler também.

F4 — Cobertura de RLS real inconsistente — a maioria dos handlers depende só do filtro manual

MÉDIA

Categoria	1. Banco sem tranca (isolamento de tenant) — observação arquitetural
Arquivo	src/lib/db/index.ts · src/lib/api/pdv-handler.ts
Linha(s)	db/index.ts:43–52 (createUnscopedDb) · pdv-handler.ts:492–495 (comentário do próprio time)

```
// db/index.ts:43-49
// Conexão HTTP simples, SEM isolamento de tenant no nível do banco — a única
// proteção é a cláusula `eq(tabela.storeId, storeId)` que cada handler escreve
// manualmente. [...]
export async function createUnscopedDb(databaseUrl, _storeId?) {
  return createDb(databaseUrl);
}
```

Descrição

O projeto tem RLS real do Postgres implementada e funcionando (drizzle/0030_ri_activate.sql, role armazix_tenant sem BYPASSRLS, createTenantDbTransactional + setTenantContext) — mas ela só é usada em um punhado de fluxos de escrita de alto risco (criar pedido, mudar status de pedido, encerrar encomenda no PDV, aplicar resultado de pagamento). TODO o restante do backend (a esmagadora maioria das ~150 rotas — produtos, categorias, clientes, cupons, balanços, sessões de caixa, impressoras, setores, planos de pagamento, etc.) usa createDb/createUnscopedDb, que não aplica RLS nenhuma — a única proteção é a cláusula eq(storeId) que cada desenvolvedor precisa lembrar de escrever em cada query, para sempre.

Por que é explorável

Não é uma falha ativa confirmada: percorri manualmente ~20 handlers de escrita/leitura neste ciclo de auditoria e TODOS incluem o filtro por storeId corretamente. O risco é estrutural: o filtro manual falha aberto — um WHERE esquecido em uma query nova (ou numa refatoração futura) vaza dado entre lojas silenciosamente, sem nenhuma rede de segurança no nível do banco para barrar. RLS real falha fechado — mesmo que o desenvolvedor esqueça o filtro, o Postgres nega a linha por conta própria.

Condições de exploração

Não é explorável hoje da forma que foi auditada (nenhum filtro ausente encontrado). É um risco latente que se materializa na PRÓXIMA query escrita sem o cuidado que o padrão atual exige em toda linha.

Impacto

Nenhum no estado atual verificado. Risco é sobre manutenção futura: cada novo handler de listagem/relatório é, por padrão, um candidato a vazamento entre lojas até que alguém releia manualmente o WHERE.

Recomendação

Expandir o uso de createTenantDbTransactional (RLS real) para as rotas de leitura/escrita de maior sensibilidade (financeiro, clientes, relatórios) de forma incremental, priorizando as adicionadas mais recentemente (reports-handler.ts, por exemplo, é 100% createUnscopedDb). Onde a migração para RLS não for viável a curto prazo, considerar um helper de query que injete eq(storeId) automaticamente em vez de exigir que cada handler repita a cláusula manualmente.

F5 — Oráculo de enumeração de PII por telefone no checkout (mitigado, residual)

BAIXA

Categoria	3. IDOR / exposição de dado (relacionado)
Arquivo	src/lib/api/crud-handler.ts
Linha(s)	checkCustomerByPhoneHandler (GET /api/customer/check, público)

```
export async function checkCustomerByPhoneHandler(request) {  
  // sem auth — rota pública  
  const storeId = url.searchParams.get("storeId");  
  const rawPhone = url.searchParams.get("phone");  
  ... retorna { exists, customer: { id, name, phone }, addresses } se achar
```

Descrição

Endpoint público (sem autenticação) que, dado um storeId e um telefone, confirma se existe um cliente com aquele telefone na loja e devolve nome + telefone + endereços de entrega. É um oráculo de PII por design — existe para pré-preencher o checkout quando o cliente já é conhecido da loja.

Por que é explorável

Já é uma decisão de produto deliberada e documentada no próprio código (comentário explícito reconhecendo o trade-off) e JÁ MITIGADA: a rota está no tier de rate limit "sensitive" (api-handler.ts, ratelimitConfigs) — o mesmo tier usado para check-email, criado especificamente para "evitar varredura em massa de PII sem exigir credencial". A projeção de campos também já exclui CPF/e-mail/avatarUrl, devolvendo só o mínimo necessário para o checkout.

Condições de exploração

Um atacante paciente, com muitos números de telefone candidatos e disposto a operar dentro do limite de taxa (ou rotacionar IPs), ainda consegue confirmar lentamente quais números são clientes de uma loja específica.

Impacto

Baixo — confirma só existência de cadastro + nome + endereço para quem já sabe o número de telefone exato; não expõe senha, CPF ou dado financeiro.

Recomendação

Nenhuma ação obrigatória — registrar como risco residual aceito. Se o volume de tráfego crescer, considerar CAPTCHA ou um limite por-IP mais agressivo especificamente nesta rota, ou exigir também os 4 últimos dígitos do CEP como segundo fator de correspondência.

Tabela consolidada de achados

Severidade	Arquivo:linha	Descrição
ALTA	src/lib/api/print-handler.ts 65–120 (núcleo: 104–107)	F1. SSRF autenticado no teste de impressora de rede
MÉDIA	src/routes/admin/relatorios.tsx relatorios.tsx:88, 60, 64, 73 · reports-handler.ts (todos os 8 handlers)	F2. Gate de permissão dos Relatórios é decorativo — sem checagem equivalente no backend
MÉDIA	src/lib/auth/email.ts email.ts:108 (template) · store-invite-handler.ts:82 · store-handler.ts:294	F3. HTML injection em e-mails transacionais (convite de equipe)
MÉDIA	src/lib/db/index.ts db/index.ts:43–52 (createUnscopedDb) · pdv-handler.ts:492–495 (comentário do próprio time)	F4. Cobertura de RLS real inconsistente — a maioria dos handlers depende só do filtro manual
BAIXA	src/lib/api/crud-handler.ts checkCustomerByPhoneHandler (GET /api/customer/check, público)	F5. Oráculo de enumeração de PII por telefone no checkout (mitigado, residual)

Recomendações priorizadas

P1**Bloquear SSRF no fluxo de teste de impressora (F1)**

Restringir `sendViaTcp()` a faixas de IP privadas/allowlist conhecida e recusar `loopback/link-local/multicast`; aplicar o mesmo filtro na gravação de `printers.path`. Adicionar `rate-limit` dedicado às 3 rotas de impressão.

P1**Fechar o gate de permissão dos Relatórios de ponta a ponta (F2)**

Implementar `usePermissaoUsuario()` com o papel real do usuário e replicar a mesma checagem dentro de cada handler de `reports-handler.ts`.

P2**Escapar HTML em todos os templates de e-mail (F3)**

Adicionar uma função de escape central e aplicá-la a `name/storeName/roleLabel` em `email.ts`; sanitizar também na entrada (`inviteStoreUserHandler`, `updateStoreHandler`).

P2**Expandir cobertura de RLS real para além do checkout (F4)**

Migrar incrementalmente os handlers de maior sensibilidade (financeiro, clientes, relatórios) para `createTenantDbTransactional`, começando por `reports-handler.ts`.

P3**Registrar o oráculo de PII por telefone como risco aceito (F5)**

Documentar formalmente a decisão; revisitar com CAPTCHA/2º fator só se o volume de tráfego justificar.

Issues para o GitHub

Uma issue em Markdown por achado acionável (F1–F4), pronta para copiar e colar. F5 foi registrado como risco aceito na seção de recomendações e não gera issue própria.

--- ISSUE 1 ---

```
# [Segurança] SSRF autenticado no teste de impressora de rede (test-raw / print-test / print-order)

**Labels sugeridas:** `security`, `alta`

## Descrição do problema
`POST /api/printers/test-raw` aceita `body.path` (IP ou hostname) de qualquer usuário autenticado com acesso a alguma loja – inclusive o papel mais baixo – e abre uma conexão TCP bruta para esse destino via `net.createConnection` (habilitado pelo `compatibility_flags: ["nodejs_compat"]` do Worker), sem nenhum bloqueio de IPs privados/loopback/link-local. O mesmo primitivo é alcançável em dois passos por `POST /api/printers/create` (o campo `path` não é validado) seguido de `print-test`/`print-order` com `send:true`.

## Por que é explorável
A resposta devolve se a conexão teve sucesso, deu timeout (5s) ou foi recusada – um oráculo clássico de SSRF/port-scan da rede que o Worker enxerga, disponível a até 60 chamadas/minuto (tier de rate limit padrão "api", já que a rota não tem entrada dedicada em `rateLimitConfigs`).

## Evidência
`src/lib/api/print-handler.ts:104-107`
```ts
const net = parseNetworkPath(body.path.trim());
if (net) {
 await sendViaTcp(net.host, net.port, escpos); // conecta em QUALQUER host:porta
}
```
`src/lib/api/printers.ts:92,153` – `path` gravado sem validação de destino.

## Impacto
Reconhecimento/varredura de rede interna acessível ao Worker; uso da infraestrutura da Armazix como relay para sondar hosts de terceiros.

## Sugestão de correção
Validar `body.path`/`printer.path` contra uma allowlist de faixas privadas conhecidas do parque de impressoras, ou no mínimo bloquear explicitamente loopback (127.0.0.0/8), link-local (169.254.0.0/16), multicast e faixas reservadas antes de `sendViaTcp`. Adicionar rate-limit dedicado e restritivo às 3 rotas de impressão (`test-raw`, `print-test`, `print-order`).

## Critérios de aceite
- [ ] `sendViaTcp`/`parseNetworkPath` rejeitam explicitamente 127.0.0.0/8, 169.254.0.0/16, faixas multicast/reservadas
- [ ] `createPrinterHandler`/`updatePrinterHandler` aplicam a mesma validação de destino ao gravar `path`
- [ ] Teste automatizado cobre a rejeição de um IP de loopback e de um link-local
- [ ] `rateLimitConfigs` ganha uma entrada restritiva para `/api/printers/test-raw`, `/api/printers/print-test` e `/api/printers/print-order`
```

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

```
# [Segurança] Gate de permissão dos Relatórios é decorativo – sem checagem equivalente no backend

**Labels sugeridas:** `security`, `média`

## Descrição do problema
`usePermissaoUsuario()` em `relatorios.tsx` sempre retorna `"admin"`, então o
filtro de UI que deveria esconder relatórios sensíveis (`aud-002` "Logs de
Alterações Críticas" → `permissao: ["admin"]`; `fin-005` "Lucro Bruto e
Líquido" → `["admin","gerente"]`; `fin-001` "Fluxo de Caixa" →
`["admin","gerente","financeiro"]`) nunca esconde nada de ninguém. Os 8
handlers reais em `reports-handler.ts` só chamam `requireStoreAccess`
(qualquer papel da loja), sem nenhuma checagem de papel/permissão.

## Por que é explorável
Qualquer funcionário autenticado da loja – inclusive "operador" – pode
chamar `GET /api/reports/logs-criticos` ou `/api/reports/fluxo-caixa`
diretamente e receber dado que o próprio catálogo já modela como restrito.

## Evidência
`src/routes/admin/relatorios.tsx:88`
```ts
function usePermissaoUsuario(): Permissao { return "admin"; }
...

`src/lib/api/reports-handler.ts` (todos os 8 handlers) – só
`requireStoreAccess(auth)`, nenhuma checagem de papel.

Impacto
Exposição de fluxo de caixa, margens de lucro e log de auditoria de ações
críticas para papéis que o dono da loja não pretendia autorizar.

Sugestão de correção
Implementar `usePermissaoUsuario()` lendo o `storeRole` real do usuário
logado, e replicar a mesma checagem de papel dentro de cada handler de
`reports-handler.ts`, reaproveitando o sistema de `roleProfiles`/
`ALL_PERMISSION_KEYS` já usado em `permissions-handler.ts`.

Critérios de aceite
- [] `usePermissaoUsuario()` retorna o papel real do usuário autenticado, não um valor fixo
- [] Cada handler de `reports-handler.ts` rejeita (403) um papel fora da lista `permissao` declarada
no catálogo para aquele relatório
- [] Teste automatizado cobre um usuário "operador" recebendo 403 em `/api/reports/logs-criticos` e
`/api/reports/fluxo-caixa`
```

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

```
[Segurança] HTML injection em e-mails transacionais (convite de equipe / nome da loja)

Labels sugeridas: `security`, `mídia`

Descrição do problema
Os templates de e-mail em `src/lib/auth/email.ts` interpolam `name` /
`storeName` direto em HTML via template literal, sem escape. O nome do
convite (`inviteStoreUserHandler`) só recebe `.trim()`, sem
`sanitizeString()`; o nome da loja pode ser trocado a qualquer momento via
`updateStoreHandler` também sem sanitização – apesar do fluxo de cadastro
original passar por `sanitizeString()`.

Por que é explorável
Um dono/admin de loja (autenticado) pode definir o nome da loja e/ou o nome
de um convite como HTML/CSS malicioso. O e-mail resultante, enviado pelo
domínio real armazix.com.br via Resend, chega para um terceiro (o
convidado, que pode nem ter conta) com esse HTML não escapado embutido.

Evidência
`src/lib/auth/email.ts:108`
```ts
<p>Olá, <strong>${name}</strong>! A loja <strong>${storeName}</strong> convidou você...</p>
```
`src/lib/api/store-invite-handler.ts:82`
```ts
const name = body.name?.trim(); // sem sanitizeString()
```
`src/lib/api/store-handler.ts:294`
```ts
.set({ name: body.name, ... }) // nome da loja gravado sem sanitização
```

Impacto
Phishing/defacement dentro de um e-mail legítimo da própria Armazix –
sobrescrever a estrutura do e-mail (ex: o botão "Aceitar convite") para
apontar para um link diferente do real.

Sugestão de correção
Adicionar uma função de HTML-escape central em `email.ts` e aplicá-la a todo
valor interpolado nos 3 templates, além de sanitizar na entrada
(`inviteStoreUserHandler`, `updateStoreHandler`) com `sanitizeString()` ou
equivalente.

Critérios de aceite
- [] `email.ts` escapa `&`, `<`, `>`, `"`, `` em todo valor interpolado nos 3 templates
- [] `inviteStoreUserHandler` sanitiza `body.name` antes de gravar/enviar
- [] `updateStoreHandler` sanitiza `body.name` antes de gravar em `stores.name`
- [] Teste automatizado cobre um nome contendo `<img onerror>` chegando escapado no HTML final do
e-mail
```

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

# [Segurança] Cobertura de RLS real inconsistente – maioria dos handlers depende só do filtro manual por storeId

**\*\*Labels sugeridas:\*\*** `security`, `média`, `débito-técnico`

#### ## Descrição do problema

O projeto tem RLS real do Postgres funcionando (`drizzle/0030\_rls\_activate.sql`, role `armazix\_tenant` sem BYPASSRLS, `createTenantDbTransactional` + `setTenantContext`), mas só é usada em poucos fluxos de escrita de alto risco (criar pedido, mudar status, encerrar encomenda no PDV, aplicar pagamento). Todo o resto do backend usa `createDb`/`createUnscopedDb`, sem RLS – a única proteção é a cláusula `eq(storeId)` que cada query precisa incluir manualmente, para sempre.

#### ## Por que é relevante

Não é uma falha ativa confirmada nesta auditoria – os ~20 handlers lidos linha a linha incluem o filtro corretamente. O risco é estrutural: o filtro manual falha **\*\*aberto\*\*** (um WHERE esquecido numa query nova vaza dado entre lojas silenciosamente); RLS real falha **\*\*fechado\*\*** (o Postgres nega a linha mesmo que o desenvolvedor esqueça o filtro).

#### ## Evidência

```
`src/lib/db/index.ts:43-49`
`...`
```

```
// Conexão HTTP simples, SEM isolamento de tenant no nível do banco – a única
// proteção é a cláusula `eq(tabela.storeId, storeId)` que cada handler escreve
// manualmente.
`...`
```

```
`src/lib/api/pdv-handler.ts:492-495` – comentário do próprio time confirmando
que só o handler mais novo do arquivo usa RLS real; o resto usa BYPASSRLS.
```

#### ## Impacto

Nenhum confirmado no estado atual. Risco latente sobre manutenção futura – cada handler novo de listagem/relatório nasce sem rede de segurança no banco.

#### ## Sugestão de correção

Migrar incrementalmente os handlers de maior sensibilidade (financeiro, clientes, relatórios – a começar por `reports-handler.ts`, que é 100% `createUnscopedDb`) para `createTenantDbTransactional`. Onde não for viável a curto prazo, avaliar um helper de query que injete `eq(storeId)` automaticamente.

#### ## Critérios de aceite

- [ ] `reports-handler.ts` migrado para `createTenantDbTransactional` + `setTenantContext`
- [ ] Lista de handlers ainda em `createUnscopedDb` documentada e priorizada
- [ ] Decisão registrada (ADR ou comentário) sobre o padrão-alvo para novos handlers

--- FIM ISSUE 4 ---